Handoff 35: File fixtures (master run WD4, ENG-08)

Date: 2026-11-02 (America/Toronto)
Written by: Claude Code (implementer), in a separate git worktree
For: the human maintainer, and ChatGPT or Claude chat (reviewer) without repository access
Status: **in_review**: one pull request (#40), not merged, not deployed. **No migration, no new dependency, no lesson uses it yet.** Includes one bug fix that applies to every check today (section 5).

> **How to use this file:** upload it with `docs/plan/browser-checks.md` and handoffs 11, 29 and 34. The pull request diff is the source of truth (`build.md` section 14).

Starter prompt for the chat assistant

```
You are reviewing file fixtures for Python Academy. Learner Python runs in a
Pyodide worker in the learner's browser, and a check case can now give the
program files in Pyodide's in-memory filesystem. Read the "File fixtures" part
of docs/plan/browser-checks.md and the worker's freshFiles(). Check that a path
cannot leave the program's folder, that nothing carries over between cases, that
no hidden expected output reaches the browser, and that the page's wording is true.
```

Run plan details

Field	Value
Stage	WD4, ENG-08
Base	`main`, at `ac167e3`, then merged again at `1947f7a` after #39, #41 and #42 merged
Part 2 boxes relied on	**None**
Stages passed over	WD3 needs WD2 and this one, both unmerged, and chaining needs B-R1

1. Step 0

`git fetch --all`, `gh pr list --state all`: #39 open, everything before it merged. No file fixture work on any branch. Worktree `../python-academy-files`; the main folder was not switched.

2. Handoff record (build.md template)

Field	Value
Task	A check case or worked example can give a program files, in the in-memory filesystem, with a clear note that they are temporary
Human goal	Units 7 and 9 can teach files fairly, and a check is never affected by what an earlier case wrote
Non-goals	**A learner's own second file** (question 1); saving anything; a lesson; checking what a program wrote, other than by what it prints
Data and compatibility	No migration. Optional `files`, sent only when set, so every existing plan and lesson view is unchanged
Deployment status	Not requested

Files changed

File	Why
`frontend/public/python-worker.js`	`freshFiles()`: empties the folders, goes home and writes the files, before each case and in Try it
`frontend/public/check-runner.js`	Passes a case's files to the worker
`frontend/app/adventure.tsx`, `style.css`	Sends the example's files on Try it, and shows the files with the "temporary" note
`backend/app/content.py`, `browser_checks.py`	`files` on cases and examples; validation; the plan and lesson view; `FileNotFoundError` as a hint tag
`backend/tests/case_runner.py`	A folder of its own per case, as the worker does
`backend/tests/test_file_checks.py` (new, 28 tests), `frontend/tests/file-checks.spec.ts` (new, 17 tests), `frontend/tests/fixtures/file-checks-api.json`, `backend/tests/fixtures/file_checks/` (new)	Section 6
`docs/plan/browser-checks.md`	The rules, and one corrected table row
`docs/run-log.md`, this handoff and its PDF	The record

3. Security note

- **Paths are checked when content loads.** Relative only, no `..`, and each part starts with a letter or digit. Authors write these paths, not learners, but this keeps a mistake inside the program's folder.

- **Everything stays in Pyodide's in-memory filesystem,** in the learner's browser. Nothing is sent to the server or saved. The worker's Content-Security-Policy is unchanged.
- **A hidden case's files reach the browser,** because the program has to read them. The page never shows them, but someone reading network traffic could. This is the same as hidden inputs today, and part of the open question from handoffs 29 and 30 about hidden data. **Hidden expected output still never leaves the server.**

4. Decisions and why

Decision	Options considered	Reason	Who decided
Fixtures now; a learner's own second file later	Both in one PR	The second file needs a second editor, a change to what the page sends, and a rule that depends on #39. This PR is already whole without it	Implementer (**assumption 1**)
Up to 10 files of 10,000 characters	Larger	Enough for the planned lessons, and it matches the output limit	Implementer (**assumption 2**)
Empty `/home/pyodide` and `/tmp`, leaving `/lib` alone	A new folder per case	A new folder would leave old files reachable by path	Implementer
The challenge shows one visible case's files as a sample	All, or none	A learner needs to see the format, and it is honest about there being others	Implementer (**assumption 3**)
Wording: "Files here are temporary…"	Say nothing	Plan Part 9 asks for it	Plan
`FileNotFoundError` allowed as a hint tag	Leave it out	It will be the most common error in these lessons	Implementer

5. Bugs found and how they were fixed

Bug or near-miss	How it was found	Fix
On `main`, a file written in one check case was still there in the next, although `browser-checks.md` said "State between cases: None". A program could change its answer between cases. No lesson uses files, and a learner could only affect their own result, so this is a correctness bug and not a leak	Reading the worker, then break B1, which puts that behaviour back and fails 3 tests	The worker cleans up before every case, whether or not the case has files
A constant was used before it was defined	First test run	Moved
A hidden file's lines could have appeared in a hidden case's description	Writing the validation	Validation refuses it

Bug or near-miss	How it was found	Fix
My tests used `import os`, which #39 refuses	Thinking through the combination	They reach `os` through `random`, which works before and after #39
Two test mistakes: a hint tag for the wrong case, and a locator that matched twice	The tests failed	Fixed

6. How it was tested

Commands and results (each suite alone, macOS, Python 3.12.14, Node 24)

Command	Result
`cd backend && DATABASE_URL=sqlite:///<temp>.db ../../python-academy/backend/.venv/bin/python -m pytest -q`, on `1947f7a`	**833 passed, 11 skipped**
Same with `TEST_POSTGRES_URL=<local PostgreSQL 16 from pgserver>`	**844 passed**
`cd frontend && npx tsc --noEmit && npm run build`	OK
`cd frontend && CI=1 npx playwright test`	**296 passed**
axe-core 4.10.3 on the lesson's cards 2, 4 and 5, at 1280 px and 375 px, once animations finish	**0 violations**

What the tests cover

Test	Where
Reading given files, each case its own, in folders	`file-checks.spec.ts`, real Pyodide
Nothing carries over: written files, new folders, `/tmp`, changed files, the current folder	Same
Python's own library survives the clean-up; a missing file gives `FileNotFoundError` on its line; the output limit still works	Same
On the page: the note, Try it with a file, a sample with no hidden file, pass, needs_work with the tagged hint, a hidden failure, a missing file, a phone	Same
Unsafe paths refused, ordinary ones allowed; too many or too large; a description quoting a hidden file	`test_file_checks.py`
The plan and view send files only when set, and existing plans and views are unchanged; hidden expected output never sent	Same
The test runner gives each case only its own files	Same
Recordings equal the real API's answers	Same

Test	Where
Every earlier suite, including `unit1-unchanged.spec.ts`	Passing, unchanged

Mocks: page tests use recorded real answers, and only for an exactly matching request. Python, the worker and the page are real. The lesson is a test fixture, never shown to learners.

Deliberate-break controls (each undone; files matched saved copies byte for byte afterwards)

What I broke	Tests that failed
B1. The clean-up between cases is removed, as on `main`	**3**
B2. `/tmp` is not cleaned	**1**
B3. A case does not go back to its folder	**1**
B4. Any file path is accepted	**11**
B5. The runner drops a case's files	**6**
B6. The sample shows a hidden case's files	**2**
B7. Try it gets no files	**1**
B8. A hidden description may quote a file	**1**

7. Known limitations

- **No learner's own second file,** so `py.own-modules` cannot be built yet.
- **A check can't look at a written file directly.** A lesson has the program print what it wrote, or read it back.
- **Files are text only.**
- **Not tested on Safari or Firefox, or with a real screen reader.**

8. Merging with #39

39 merged first, so I merged `main` into this branch. The 7 conflicted files were all places where both add a line. The code matches the combination tested earlier.

9. Live checks for you

No real lesson has files yet, so there is nothing new to see. Please check that an ordinary lesson in units 2 and 3 works as before, and that there are **no CSP reports** in DevTools → Console.

10. Questions for you

1. **A learner's own second file:** answered by the owner on 2026-11-02: its own stage before unit 7, and the import guard must apply inside learner modules. Proposed ID: ENG-08b.
2. **Assumption 2, the limits.** *Recommended, not yet confirmed: keep them.*
3. **Assumption 3, showing a sample.** *Recommended, not yet confirmed: keep it.*
4. **Hidden data in the browser:** answered by the owner on 2026-11-02. It may reach the browser, and hidden expected results may not.

11. For the docs sync

Where	What
Backlog ENG-08	proposed → in_review (#40, handoff 35), noting the second file is not built
Backlog, maybe a new row	The state-between-cases fix
Stage table	"35. ENG-08 | File fixtures | in_review (#40, handoff 35)"
Decision log	A chapter from sections 3 to 6

12. Next concrete action

Review and merge.

13. Removing the worktree safely

From the **main** folder, after this PR is merged:

```
cd /Users/DEV/Projects/python-academy
git -C ../python-academy-files status --short   # expect nothing, or only ??
frontend/AGENTS.md
rm -f ../python-academy-files/frontend/AGENTS.md   # generated by Next.js, not part of the
project
git worktree remove ../python-academy-files     # deletes only that folder; refuses if
anything is uncommitted
git worktree prune
```

```
git branch -d eng/file-fixtures                 # refuses if the branch is not merged
git branch -D tmp/files-on-imports              # a local test branch of the combined
result, never pushed
```

Don't delete the folder in the Finder or with `rm -rf`; Git would keep a stale record of it (`git worktree prune` fixes that). If `git branch -d` refuses after a squash merge, check the PR says "Merged" on GitHub, then use `git branch -D eng/file-fixtures`.